

AntiSamy

OWASP's AntiSamy project secures web applications by ensuring that there is no malicious code being supplied by clients using their service

Pwn2Own

This yearly hacking contest requires hackers to exploit browsers and devices for a cash prize, and the device they hacked.



Firefox protecting from malware

Well, because you fell for it. To make a loose analogy, you wouldn't just go to an address because a friend of a friend's elder brother's friend's friend said they have a good selection of boots. However we indiscriminately click on suspicious links that could take us anywhere.

Browsers do not come with inbuilt phishing and malware filters that check the URL you are visiting against a database of malicious URLs, and blocks access to entries that match. Most scan URLs that start circulating on the internet end up here sooner or later.

Google's malware and phishing databases are used by Mozilla Firefox, Safari and Google Chrome (obviously), while both Internet Explorer and Opera use data from Netcraft which has tested to be superior to Google's data.

Sandboxing

Currently this technology is only properly implemented in Google Chrome and Internet Explorer.

Google Chrome creates separate process for the main browser, extensions and tabs such that nearly all tabs run in a separate process (tabs from the same site might run in the same process). Additionally, web sites run in a sandboxed process that has limited access to the computer's resources. As such, even if a browser exploit in a tab were to succeed, it would fall to deaf ears as the tab itself doesn't have the privileges to execute the exploit code. Google Chrome also sandboxes the Flash Player,

extending security to the plugin.

Internet Explorer introduced a new protected mode with IE7 that ran the entire browser in the sandbox, because of this trying to open a local folder in the browser resulted in the browser opening a new window. IE8 added multiple processes so that each tab could run in a separate process and

thus individual tabs could be in protected mode (sandboxed) while others could point to local folders.

While running a sandboxed browser you could open malicious links, and probably your computer would still not be damaged, don't do it though, that video of a monkey sniffing its rear is not worth it.

Content security policy

The sheer brilliance of the web lies in being able to mash up data coming from different sources on the internet. This however also poses numerous risks, as many web sites have malicious content. An insecure web site can be vulnerable to SQL injection, CSRF (Cross-site request forgery) and XSS (Cross-site scripting) attacks.

Mozilla's new Content Security Policy allows web sites to specify the domains from which the web sites can receive scripts, and only scripts from those domains will be executed. Web sites can even globally disable scripting if their site doesn't need it.

Content Security Policy doesn't immediately make a web site secure, it merely adds another layer of security that increases the difficulty of hacking a web site. Currently it is only present in Mozilla Firefox 4 and in implemented in few sites (including Twitter), but support for it is bound to increase.

Security and you!

Browser vendors are coming up with increasingly intelligent methods of

Anatomy of a flaw

Not all flaws are user errors of course; often browsers (or the components that they rely on) have faults that leave the system compromised. Let us take a look at one such flaw that affected a number of browsers:

Title: Remote code execution using malformed PNG image

Description: "OUSPG researcher Aki Helin reported a buffer overflow in Mozilla graphics code which consumes image data processed by libpng. A malformed PNG file could be created which would cause libpng to incorrectly report the size of the image to downstream consumers. When the dimensions of such images are underreported, the Mozilla code responsible for displaying the graphic will allocate too small a memory buffer to contain the image data and will wind up writing data past the end of the buffer. This could result in the execution of attacker-controlled memory."

Here such an image could be posted in a social networking website and might be viewed by someone without realizing what is happening in the background. This image could cause a crash, or worse allow an attacker to compromise your system.

While what we have printed above is from a Mozilla Foundation Security Advisory (MSFA2010-41), this issue affected many applications using libpng, including Chromium.

securing their browsers, but as they say, built a more idiot-proof system, and bigger better idiots will appear.

Browser security will always be a close battle, with newer threats emerging continuously and new methods of stopping them coming soon. Every once in a while, browser developers will figure out a type of attack before it is ever used, but usually there are casualties.

Computers are thrust before people long before they have an understanding of what is possible and the implications of their actions. Clicking on links seems harmless enough, and there isn't always a clear feedback like the one a child receives after putting his hand on a burning object.

The answer as always lies in education. Tech yourself, teach your friends how to be safe on the web, and to use their browser's security features to their advantage.

Against stupidity, the gods themselves contend in vain — Friedrich Schiller. 